

## 🎯 本章目标

掌握 C++ 的属性(attribute)机制:标准属性 `nodiscard`、`deprecated`、`fallthrough`、`maybe_unused`、`noreturn`、`likely/unlikely` 的用途与写法;理解 GCC/Clang 的 `__attribute__` 与 MSVC 的 `__declspec` 扩展;学会用兼容性宏与条件编译写出跨编译器、跨平台的可移植代码。

## 📦 前置知识

熟悉函数声明、`switch` 语句(第1、3章);了解预处理宏(第10章)。本章代码基于 C++17(属性语法 C++11 起),`likely/unlikely` 为 C++20,编译器扩展部分按 GCC/Clang 语法展示并给出 MSVC 对照。

|             |               |               |
|-------------|---------------|---------------|
| §12.1 属性与注解 | §12.2 标准属性全览  | §12.3 属性的契约价值 |
| §12.4 编译器扩展 | §12.5 与 C 的对比 | §12.6 工程实践    |

## §12.1 属性是什么:给编译器的契约

C++ 的属性(attribute)是写在双中括号里的标记:`[[nodiscard]]`、`[[deprecated("原因")]]`、`[[fallthrough]]`,放在函数、变量、类型或语句之前,向编译器声明某种契约。属性不引入新的运行时行为——它要么是编译期检查的开关(`nodiscard` 丢弃返回值就报警),要么是给实现的信息(`deprecated` 的弃用警告),要么是让程序员意图显式化(`fallthrough` 声明穿透是有意的)。与 C#/Java 的注解不同,那里的注解是运行时可查的元数据,可以驱动框架行为;C++ 的属性基本是“编译期的话”,运行时不留下痕迹。

属性语法有三个要素:名字(标准属性名或命名空间限定名)、可选参数(括号里的字符串等)、位置(函数前、返回类型后、语句前等)。编译器遇到不认识的属性应当忽略并给出警告(标准要求实现可以忽略未知属性)——这给了扩展空间,也让拼错名字的后果变得隐蔽:属性不生效只是少一次检查,不报错。工程上因此要配合 `-Werror` 之类的严格编译选项,让属性错误显形。

属性在工具链里的角色也要说清:clang-tidy、GCC 的 `-Wattributes` 能检查属性写法的合法性与位置;静态分析器会读取 `nodiscard`、`deprecated` 并把发现写进报告。属性是“人写契约、编译器执行、工具链审计”三环节的中间层——契约写得越准,编译器与静态分析就越有用。把属性当作接口文档的一部分来写,而不是随手加的花哨装饰。

### 📋 清单 12-0 属性的位置

```

#include <iostream>
#include <string>

[[nodiscard]] int f(int v);           // 位置 1:函数声明前
int g(int v) [[nodiscard]] { return v; } // 位置 2:函数定义后

class [[deprecated]] OldApi { };      // 类型前:整个类型弃用

[[nodiscard]] int mustUse() { return 7; }

int main() {
    int x = mustUse();                // 正确使用
    // mustUse();                     // 警告:丢弃返回值
    std::cout << x << "\n";
    return 0;
}

```

## §12.2 标准属性全览

标准属性数量不多,个个有用。[[nodiscard]] 声明"返回值不得丢弃":调用方忽略返回值时编译器报警,适合错误码、句柄、资源类、以及一切"结果必须被处理"的函数;C++17 起可以带原因文字 [[nodiscard("必须处理返回值")]],报警时一并显示。[[deprecated]] 标记弃用,并附迁移建议;调用弃用接口产生警告,让旧接口的存量调用逐步现形,是库演进的温和工具。

[[fallthrough]] 用在 switch 的 case 末尾,声明"这里故意穿透到下一个 case",抑制"穿透警告"并给读者明确信号——没有它,编译器(与同事)都默认穿透是疏忽。[[maybe\_unused]] 声明"这个实体可能用不到",抑制未使用警告,适合调试参数、条件编译产物与占位形参。[[noreturn]] 声明函数不会返回(如 terminate、自定义错误处理),让编译器知道函数之后的代码不可达,顺带优化。C++20 的 [[likely]] 与 [[unlikely]] 给分支预测提示:告诉编译器哪个分支是热路径,对性能敏感的分支判断有实际收益。

### 清单 12-1 标准属性全家福

```

#include <iostream>
#include <string>

// nodiscard: 丢弃返回值必报警,带原因文字
[[nodiscard("结果必须被处理")]]
int check(int v) {
    return v > 0 ? 0 : -1;
}

// deprecated: 旧接口弃用,提示迁移方向
[[deprecated("改用 check()")]]
int legacyCheck(int v) { return v > 0 ? 0 : -1; }

// noreturn: 函数永不返回
[[noreturn]] void fatal(const std::string& msg) {
    std::cerr << "致命错误: " << msg << "\n";
    std::exit(1);           // 之后代码不可达
}

// maybe_unused: 这个参数某些编译配置下用不到
void logAt([[maybe_unused]] int verbose, const std::string& msg) {
    if (verbose > 0) std::cout << msg << "\n";
}

int classify(int v) {
    switch (v) {
    case 0:
        return 0;
    case 1:
        [[fallthrough]];           // 故意穿透: 1 与 2 同结果
    case 2:
        return 1;
    default:
        return -1;
    }
}

int main() {
    if (check(5) == 0) std::cout << "通过\n";
    // check(5);                  // 警告: 丢弃了带 nodiscard 的结果
    // legacyCheck(5);            // 警告: 调用已弃用接口
    std::cout << classify(1) << " " << classify(2) << "\n";
    logAt(0, "静默模式");
    return 0;
}

```

| C 的写法         | C++ 的写法                  | 说明                     |
|---------------|--------------------------|------------------------|
| case 穿透靠运气与注释 | case 1: [[fallthrough]]; | C 穿透是默认行为,C++ 要求显式声明意图 |
| 错误码调用点检查靠自觉   | [[nodiscard]] 编译器强制      | 契约从注释升级为编译期检查          |
| 弃用接口直接删除或留着   | [[deprecated]] 温和过渡      | 存量调用先报警后清理,不破接口        |

## §12.3 属性的契约价值:nodiscard 的工程意义

属性里工程价值最高的是 `[[nodiscard]]`, 它的本质是把"必须使用返回值"从口头约定变成编译期强制。C 时代, 错误码函数的手册上写着"调用者必须检查返回值", 但编译器不管, 漏检查的代码静默通过, 错误悄悄吞掉——这是 C 程序里最隐蔽的一类 bug。加上 `[[nodiscard]]` 之后, 所有调用点漏检查都会报警, 配合 `-Werror` 直接编译失败, 错误无处遁形。这正是 C 程序员最该尽快用上的 C++ 能力之一。

应用场景清单: 返回错误码的函数(必查)、返回资源句柄/智能指针的函数(必接管, 否则泄漏)、返回新分配的对象的工厂函数、以及"返回值是唯一出口"的查询函数。标准库带头示范: `std::make_unique`、`std::vector::empty`、`std::map::insert` 等在新标准中都被标记了 `nodiscard`。库作者给公共 API 的每个"结果型"函数都加 `nodiscard`, 接口自文档化, 调用方被编译器教育着写对代码。

### 清单 12-2 `nodiscard` 的工程应用

```
#include <iostream>
#include <memory>
#include <fstream>
#include <string>

// 错误码函数: 结果必须检查
enum class Err { Ok, NotFound, Full };
[[nodiscard]] Err tryInsert(int v) {
    return v > 0 ? Err::Ok : Err::Full;
}

// 工厂函数: 指针必须被接管, 否则泄漏
[[nodiscard]] std::unique_ptr<std::string>
makeGreeting(const std::string& name) {
    return std::make_unique<std::string>("你好, " + name);
}

// 打开文件: 失败必须被处理
[[nodiscard]] std::ifstream openLog(const std::string& path) {
    return std::ifstream(path);           // C++11 起 ifstream 可移动返回
}

int main() {
    Err e = tryInsert(42);                 // 正确处理
    if (e != Err::Ok) return 1;

    auto g = makeGreeting("张三");         // unique_ptr 接管
    std::cout << *g << "\n";

    std::ifstream log = openLog("run.log");
    if (!log) {
        std::cerr << "日志打不开, 但不致命\n";    // 处理失败路径
    }
    // tryInsert(-1);                       // 错误: 丢弃错误码, 编译报警
    // makeGreeting("李四");                 // 错误: 泄漏风险, 编译报警
    return 0;
}
```

### ⚠ 易错点 1:属性位置写错,静默失效

`[[nodiscard]]` 放在返回类型之前是有效的,但和函数名之后、参数列表之前等位置混用时,编译器可能当作未知属性忽略并只给警告;函数指针、lambda、类型别名上的属性位置规则更绕。规则:属性紧跟它修饰的实体,标准写法按 `cppreference` 示例抄,别自由发挥;开 `-Wattributes` 让位置错误显形。

### ⚠ 易错点 2:nodiscard 的边界

`[[nodiscard]]` 只对"丢弃返回值"报警:void 函数加上无意义;C++20 之前的版本,先把返回值存进变量再丢弃、或 `static_cast<void>(f())` 显式忽略,都不报警——后者其实是合法的"我确实不想处理"。它拦的是疏忽,不是恶意绕过,别指望它替代代码评审。

## §12.4 编译器扩展: `__attribute__` 与 `__declspec`

标准属性之外的天地由编译器方言统治:GCC/Clang 用 `__attribute__((...))`,MSVC 用 `__declspec(...)`。常见扩展:`__attribute__((packed))` 取消结构体对齐填充,用于协议头等紧凑布局;`__attribute__((aligned(16)))` 指定对齐,适合 SIMD 数据;`__attribute__((unused))` 抑制未使用警告(标准版是 `[[maybe_unused]]`);`__attribute__((visibility("hidden")))` 控制动态库符号可见性;MSVC 的 `__declspec(dllexport/dllimport)` 是 Windows 动态库导出导入的标配。它们与标准属性的区别:能力更强、语法不同、不可移植。

不可移植意味着直接裸用,换编译器就编译失败。工程标准做法是封装成宏:CPP\_API、CPP\_PACKED 之类,内部用条件编译分别展开成各编译器的写法,业务代码只写宏。宏还要处理"未知编译器怎么办"——退化为空,保持可编译。这套兼容性封装是跨平台库的地基:Windows 下展开成 `__declspec`,Linux 下展开成 `__attribute__`,macOS 与 Linux 一致,三个平台共用一份头文件。

| C 的写法                             | C++ 的写法                           | 说明                                 |
|-----------------------------------|-----------------------------------|------------------------------------|
| <code>#pragma pack(1)</code> 全局生效 | <code>[[gnu::packed]]</code> 或封装宏 | <code>pragma</code> 影响范围含糊,属性精确到实体 |
| <code>__attribute__</code> 方言裸用   | 兼容宏 + 条件编译                        | C++ 惯例:方言藏进宏,代码可移植                 |
| 错误码检查靠自觉                          | <code>[[nodiscard]]</code> 编译器强制  | C++ 把约定变成检查                        |

清单 12-3 `packed` 协议头与兼容宏

```

#include <iostream>
#include <cstdint>

// 兼容宏:一个名字,三种平台各自的写法
#ifdef _MSC_VER
#define CPP_PACKED_BEGIN __pragma(pack(push, 1))
#define CPP_PACKED_END __pragma(pack(pop))
#elif defined(__GNUC__) || defined(__clang__)
#define CPP_PACKED_BEGIN
#define CPP_PACKED_END
#define CPP_PACKED_ATTR __attribute__((packed))
#else
#define CPP_PACKED_BEGIN
#define CPP_PACKED_END
#define CPP_PACKED_ATTR
#endif

CPP_PACKED_BEGIN
// 网络协议头:紧凑布局,无对齐填充
struct PacketHeader {
    std::uint32_t magic;        // 魔数
    std::uint16_t type;        // 消息类型
    std::uint16_t length;      // 正文长度
    std::uint8_t checksum;     // 校验和
} CPP_PACKED_ATTR;
CPP_PACKED_END

int main() {
    // packed 后:4 + 2 + 2 + 1 = 9 字节,没有填充
    std::cout << "包头大小: " << sizeof(PacketHeader) << " 字节\n";
    PacketHeader h{0x12345678, 1, 100, 0xAA};
    std::cout << "magic=" << std::hex << h.magic << std::dec
              << ", type=" << h.type << "\n";
    return 0;
}

```

### ⚠ 易错点 3:packed 的代价

取消对齐填充换来紧凑布局,代价是未对齐访问:成员横跨字边界,读取可能变慢,在部分架构(如老式 ARM)上直接是硬件错误;同时 packed 结构体不能安全地取成员地址交给需要对齐的接口。协议头用 packed 合理,热点数据结构别用,能用字段序列化就别整块打包。

## §12.5 与 C 的对比:从 pragma 到属性

C 程序员对"注解"并不陌生,只是它的形态是一堆方言:#pragma pack、#pragma GCC diagnostic、\_\_attribute\_\_((packed)),各编译器语法不一、行为不齐、没有标准。C23 标准开始引入 [[nodiscard]]、[[deprecated]]、[[fallthrough]] 等属性,语法与 C++ 对齐,但数量仍远少于 C++ 标准库的约定。C++ 的路线更彻底:C++11 起就把属性语法标准化,标准属性随语言演进持续增加,编译器扩展通过命名空间属性(如 [[gnu::always\_inline]])与标准语法统一。

迁移到 C++ 时,最该做的三件事:把 return 型错误检查的函数全部加上 [[nodiscard]];把 #pragma pack 的用法改成 C++ 属性或封装宏;把 \_\_attribute\_\_ 裸用收进兼容宏。改动量不大,收益立竿见影:编译器的检查能力立刻多了一层。至于"注解能驱动框架行为"的期待,回到上一章的结论——C++ 的注解是给编译器的,框架行为请交给模板与代码生成。

| C 的写法                       | C++ 的写法             | 说明               |
|-----------------------------|---------------------|------------------|
| #pragma pack(1) 文件级生效       | 实体级属性,作用域精确         | C++ 注解不污染上下文     |
| __attribute__((packed)) 无标准 | 标准 [...] 语法统一       | C++ 属性标准化,方言靠边   |
| 错误码函数无强制检查                  | [[nodiscard]] 编译期报警 | C23 开始跟进,C++ 已成熟 |

清单 12-4 C 的 pragma 与 C++ 的属性对照

```
/* C 的写法:#pragma 与方言,影响范围不精确 */
#pragma pack(push, 1)
struct Packet {
    unsigned char ver;
    unsigned short len;
};
#pragma pack(pop)

// C++ 的写法:属性精确到实体,方言收进宏
#define PACKED __attribute__((packed))
struct Packet {
    std::uint8_t ver;
    std::uint16_t len;
} PACKED;

// C23 起,标准属性与 C++ 对齐:
// [[nodiscard]]、[[deprecated]]、[[fallthrough]]
```

## §12.6 属性的工程实践

属性用得好不好,取决于项目纪律。四条实践:第一,严格编译选项打底——-Wall -Wextra -Werror(MSVC 对应 /W4 /WX),让属性报警升级为编译失败,否则"报警忽略"和"没写属性"没区别;第二,公共 API 的结果型函数全部 nodiscard,错误码、句柄、工厂函数一个不漏;第三,弃用走温和路线——先 deprecated 给过渡期,统计存量调用清理完再删除,别一夜之间删接口;第四,方言扩展全部收进封装宏,头文件里一个裸 \_\_attribute\_\_ 都不该有,换编译器只需改宏定义。

最后澄清一个常见误解:属性不是性能魔法。[[likely]]/[[unlikely]] 只是提示,现代 CPU 的分支预测器有自己的统计;[[noreturn]]、[[maybe\_unused]] 主要改善诊断与代码可读性。属性的价值集中在两处:让编译器替你检查约定(contract),让代码意图显式化(documentation)。把这两点用好,属性就值回票价了。

代码评审时,属性应当作为接口评审的一环:新函数有没有结果被丢弃的风险?旧接口有没有明确弃用?packed 有没有出现在不该出现的地方?评审清单里加一条"属性检查",比事后补报警要便宜得多。另外记住属性是渐进能力:C++11 的六个属性用到熟,再拥抱 C++20 的 likely/unlikely 与 C++23 的 [[assume]],不必一次学全。



## 注解与属性速查表

| 属性           | 语法示例                                  | 要点                  |
|--------------|---------------------------------------|---------------------|
| nodiscard    | [[nodiscard]] int f();                | 丢弃返回值即报警,C++17 可带原因 |
| deprecated   | [[deprecated("用新接口")]]                | 调用即警告,带迁移建议         |
| fallthrough  | [[fallthrough]];                      | 声明 switch 穿透是有意的    |
| maybe_unused | [[maybe_unused]] int x;               | 抑制未使用警告             |
| noreturn     | [[noreturn]] void die();              | 声明函数不返回,代码不可达       |
| likely       | if ([[likely]] cond)                  | C++20,热路径提示         |
| unlikely     | if ([[unlikely]] cond)                | C++20,冷路径提示         |
| 命名空间属性       | [[gnu::always_inline]]                | 编译器扩展的标准语法          |
| packed       | __attribute__((packed))               | 取消填充,协议头常用          |
| aligned      | __attribute__((aligned(16)))          | 指定对齐,配合 SIMD        |
| visibility   | __attribute__((visibility("hidden"))) | 动态库符号可见性            |
| dllexport    | __declspec(dllexport)                 | MSVC 动态库导出          |
| 兼容宏          | CPP_API / CPP_PACKED                  | 方言收进宏,代码可移植         |
| 严格编译         | -Wall -Wextra -Werror                 | 属性报警升级为编译失败         |

## 自测题

- Q 1. C++ 属性与 Java/C# 注解的本质区别是什么?**
- Q 2. [[nodiscard]] 适合哪些场景?为什么错误码函数最应该加?**
- Q 3. [[fallthrough]] 解决什么问题?不写它编译器会怎样?**
- Q 4. \_\_attribute\_\_ 与 [[...]] 有何关系?跨平台代码为什么要封装宏?**
- Q 5. 属性报警被忽略的后果是什么?工程上如何让属性检查真正生效?**

### ✓ 参考答案

- Java/C# 注解是运行时可查询的元数据,能驱动框架行为;C++ 属性是编译期的契约与提示,运行时不留下痕迹。C++ 的框架行为由模板与代码生成承担,属性只负责编译期检查与意图显式化。
- 所有"结果必须被处理"的函数:错误码、资源句柄、工厂函数、查询函数。错误码最该加,因为 C 时代漏查错误码是静默的、不可见的,加 nodiscard 后漏查直接编译报警,配合 -Werror 变成编译失败。
- switch 的 case 穿透默认被当作"忘记 break",编译器警告;[[fallthrough]] 声明穿透是有意设计,抑制警告并让读者明确看到意图。不写它会收到警告,写了又没穿透会收到另一个警告。
- [[...]] 是标准语法,\_\_attribute\_\_ 是 GCC/Clang 方言(MSVC 是 \_\_declspec)。方言不可移植,直接裸用换编译器就失败;封装成宏后业务代码统一,换平台只改宏定义。
- 报警被忽略等于属性没写:漏检查的错误码、被遗忘的弃用接口都继续静默存在。工程上开 -Wall -Wextra -Werror(/W4 /WX),属性报警全部升级为编译失败,让检查不可绕过。



## 实验:安全协议库

本实验把本章属性用在真实场景:协议库的错误码全部 `nodiscard`,旧接口走 `deprecated` 过渡,协议头解析器用 `fallthrough` 表达多字节前缀,打包函数用 `packed` 结构体,日志参数用 `maybe_unused`,致命错误用 `noreturn`,最后用兼容宏保证三平台可编译。代码分三部分,覆盖点已在注释中标明。

### 实验 12•第一部分:契约属性(覆盖:`nodiscard`、`deprecated`、`noreturn`)

```
#include <iostream>
#include <string>
#include <cstdint>
#include <cstdlib>
#include <vector>

enum class ProtoErr { Ok, BadMagic, BadLength };

// 覆盖:nodiscard 带原因——协议处理结果必须被检查
[[nodiscard("必须检查协议错误")]]
ProtoErr parseHeader(const std::vector<std::uint8_t>& data) {
    if (data.size() < 9) return ProtoErr::BadLength;
    if (data[0] != 0x78 || data[1] != 0x56) return ProtoErr::BadMagic;
    return ProtoErr::Ok;
}

// 覆盖:deprecated——旧接口过渡,提示迁移
[[deprecated("改用 parseHeader()")]]
bool parseHeaderOld(const std::vector<std::uint8_t>& data) {
    return parseHeader(data) == ProtoErr::Ok;
}

// 覆盖:noreturn——致命错误直接终止
[[noreturn]] void protocolFatal(const std::string& why) {
    std::cerr << "协议致命错误: " << why << "\n";
    std::exit(2);
}
```

### 实验 12•第二部分:`packed` 与兼容宏(覆盖:`packed`、兼容宏、条件编译)

```

// 覆盖:兼容宏—Windows 用 pragma,Linux/macOS 用 attribute
#if defined(_MSC_VER)
#define CPP_PACK_BEGIN __pragma(pack(push, 1))
#define CPP_PACK_END __pragma(pack(pop))
#define CPP_PACKED_ATTR
#elif defined(__GNUC__) || defined(__clang__)
#define CPP_PACK_BEGIN
#define CPP_PACK_END
#define CPP_PACKED_ATTR __attribute__((packed))
#else
#define CPP_PACK_BEGIN
#define CPP_PACK_END
#define CPP_PACKED_ATTR
#endif

// 覆盖:packed 协议头,9 字节紧凑布局
CPP_PACK_BEGIN
struct Header {
    std::uint32_t magic; // 0x12345678 的网络序
    std::uint16_t type;
    std::uint16_t length;
    std::uint8_t flags;
} CPP_PACKED_ATTR;
CPP_PACK_END

// 覆盖:maybe_unused—日志级别在某些构建里用不到
void debugDump(const std::vector<std::uint8_t>& data,
               [[maybe_unused]] int verbose) {
#ifdef _DEBUG
    if (verbose > 0) {
        for (auto b : data) std::printf("%02X ", b);
        std::printf("\n");
    }
#endif
}

```

## 实验 12•第三部分:解析器与 main(覆盖:fallthrough、属性综合)

```

// 覆盖:fallthrough—类型字段的连续分支有意穿透
int typeGroup(std::uint16_t type) {
    switch (type) {
        case 0x0001:           // 请求
            return 1;
        case 0x0002:           // 响应
        case 0x0003:           // 通知
            [[fallthrough]];   // 覆盖:三种通知类统一归类
        case 0x0004:
            return 2;
        default:
            return 0;
    }
}

int main() {
    std::vector<std::uint8_t> good{0x78, 0x56, 0x34, 0x12,
                                   0x01, 0x00, 0x64, 0x00, 0x00};

    // 覆盖:nodiscard—检查结果不可丢弃
    ProtoErr r = parseHeader(good);
    if (r != ProtoErr::Ok) protocolFatal("头部校验失败");

    std::vector<std::uint8_t> bad{0x00, 0x00, 0x00, 0x00,
                                   0x01, 0x00, 0x64, 0x00, 0x00};

    r = parseHeader(bad);           // BadMagic
    std::cout << "坏包结果: "
               << (r == ProtoErr::BadMagic ? "魔数错误" : "其他") << "\n";

    Header h{0x12345678, 0x0002, 100, 0x01};
    std::cout << "包头大小 " << sizeof(h) << " 字节"
               << ",组别 " << typeGroup(h.type) << "\n";

    debugDump(good, 1);             // 覆盖:maybe_unused 参数
    // parseHeaderOld(good);         // 警告:已弃用接口
    return 0;
}

```

实验覆盖点核对:nodiscard(parseHeader 的错误码强制检查)、deprecated(parseHeaderOld 过渡)、noreturn(protocolFatal)、fallthrough(typeGroup 的穿透分组)、packed 与兼容宏(Header 三平台可编译)、maybe\_unused(debugDump 参数)、属性综合应用。扩展练习:把 parseHeader 改成解析 Header 结构体并校验 magic 字段;或给 typeGroup 增加 [[unlikely]] 的 default 分支,对比开关警告。



### 工程建议 1:公共接口一次写对属性

库的公共 API 发布前过一遍属性清单:结果型函数 nodiscard,废弃接口 deprecated,平台细节 packed/导出宏。属性发布后补加会带来一波调用方编译警告,宁可写接口时一次做对,也不在 v2 里补课。



### 工程建议 2:把方言全部收进头文件宏

项目里建立一个 portability.h,集中定义 CPP\_API、CPP\_PACKED 等兼容宏,业务代码只 include 不裸用方言。换编译器、加平台时的改动收敛到一个文件,这是跨平台工程的最低成本方案。

## 本章小结

属性是 C++ 给编译器的契约语言:标准属性覆盖检查与意图(nodiscard、deprecated、fallthrough、maybe\_unused、noreturn、likely),编译器扩展提供平台能力(packed、aligned、dllexport),兼容宏让方言可移植。配合严格编译选项,属性把"约定"变成"检查"。下一章进入模块系统:从 #include 到 C++20 模块。